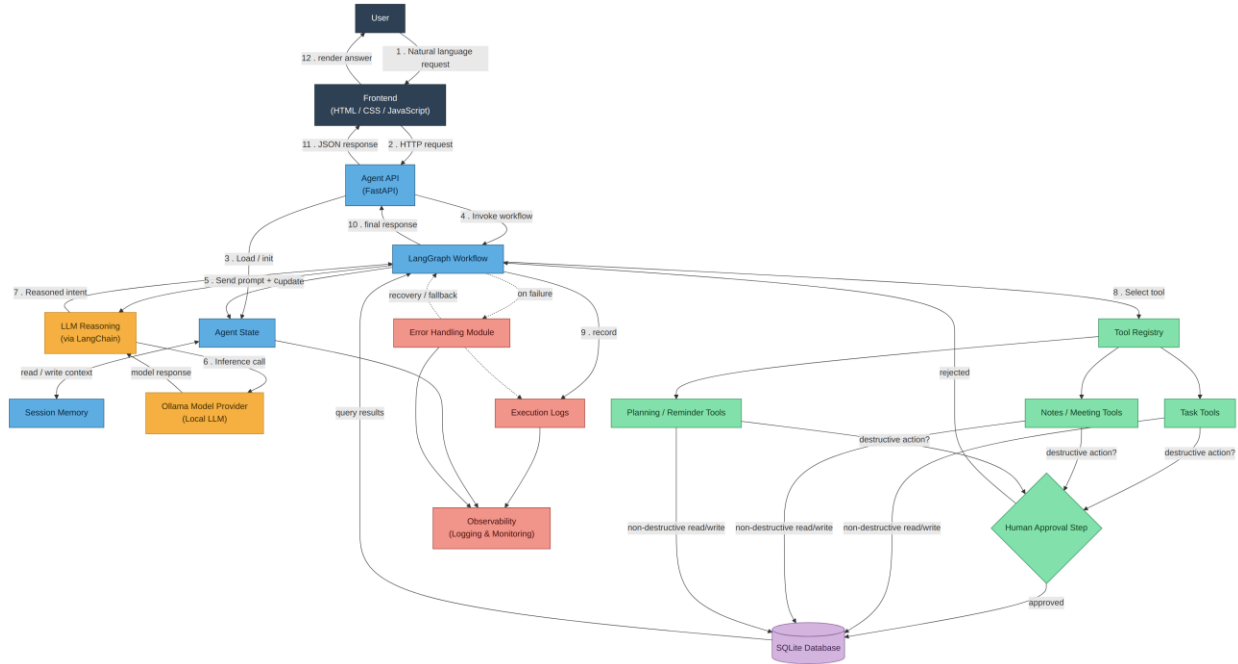


Architecture Diagram & Explanation

1. System Architecture Diagram

The diagram below illustrates the end-to-end architecture of the AI Productivity Agent, showing data flow from user input through reasoning, tool execution, human approval, persistence, and observability.



2. Component Explanations

Each component's purpose, responsibilities, inputs, and outputs are summarised below.

Component	Purpose	Responsibilities	Inputs	Outputs
User	Initiates requests.	Express task/meeting needs in natural language; approve or reject destructive actions.	Natural-language text	Rendered responses
Frontend	User interface layer.	Capture input, send HTTP requests, render agent responses and approval prompts.	User text, API responses	HTTP requests; rendered UI
Agent API	Entry point of backend.	Receive HTTP requests, validate payloads, initiate LangGraph execution, return responses.	HTTP request (JSON)	HTTP response (JSON)
LangGraph Workflow	Orchestration engine.	Sequence reasoning, tool selection, approval, and execution as a stateful graph.	Agent state, user request	Updated state; final response
Agent State	Runtime context object.	Hold current request, intent, tool results, and pending approvals across graph nodes.	Node outputs	Context for next node
Session Memory	Conversation continuity.	Persist recent dialogue turns to inform multi-turn reasoning.	Conversation turns	Historical context
LangChain	LLM integration layer.	Format prompts, parse model output, expose tool-calling interface to LangGraph.	Prompt + context	Structured intent/response
Ollama	Local model provider.	Host and execute the LLM inference locally, without external API calls.	Formatted prompt	Generated text/completion
Tool Registry	Tool catalogue.	Map reasoned intent to the correct callable tool function.	Selected intent	Tool reference/handle
Task Tools	Task CRUD operations.	Create, update, delete, complete, and search tasks.	Task parameters	Task records
Notes/Meeting Tools	Meeting management.	Create/update meetings; extract action items from notes.	Meeting/notes data	Meeting records; action items

Component	Purpose	Responsibilities	Inputs	Outputs
Planning Tools	Reminders & scheduling.	Schedule and manage reminders linked to tasks or meetings.	Item ID, time	Reminder records
Human Approval	Safety checkpoint.	Pause execution before destructive operations pending user confirmation.	Proposed destructive action	Approve/reject decision
SQLite Database	Persistence layer.	Store and retrieve tasks, meetings, and reminders.	SQL queries	Query results; write confirmations
Execution Logs	Operational record.	Record tool invocations, decisions, and outcomes for traceability.	Workflow events	Structured log entries
Error Handling	Failure recovery.	Catch LLM, tool, and database errors; trigger retries or graceful fallback messages.	Exceptions/failures	Recovery action; error message
Observability	Monitoring layer.	Aggregate logs and state metrics to support debugging and performance monitoring.	Logs, state, errors	Dashboards/alerts

3. End-to-End Workflow

The following steps describe how a single user request traverses the system, from initial input to final response:

1. The User submits a request in natural language (e.g., "remind me to submit the report by Friday") through the Frontend.
2. The Frontend sends the request as an HTTP call to the Agent API (FastAPI), which validates the payload.
3. The Agent API initialises or loads the Agent State and retrieves relevant Session Memory for conversational context.
4. The Agent API invokes the LangGraph Workflow, which becomes the central orchestrator for the remainder of the request.
5. LangGraph forwards the request and context to the LLM through LangChain, which formats the prompt and sends it to Ollama for local inference.
6. The LLM reasons about the user's intent and returns a structured decision, which LangGraph uses to select a tool from the Tool Registry (Task, Notes/Meeting, or Planning Tools).
7. If the selected action is destructive (e.g., delete task, cancel meeting), LangGraph routes execution through the Human Approval step and pauses until the user responds.
8. Upon approval (or immediately for non-destructive actions), the selected tool executes against the SQLite Database, performing the required read or write operation.
9. LangGraph updates the Agent State with the tool's results, records the operation in Execution Logs, and forwards both to Observability for monitoring; any failures are intercepted by the Error Handling Module, which attempts recovery or produces a safe fallback message.
10. LangGraph composes the final natural-language response, which the Agent API returns to the Frontend for rendering to the User.